

Local Graph-Aware Hop-by-Hop Routing for Dynamic LEO Satellite Networks

Wenxiang Zhang

School of Software Engineering
Xi'an Jiaotong University
 Xi'an 710049, China
 2060828687@qq.com

Yining Gao

School of Software Engineering
Xi'an Jiaotong University
 Xi'an 710049, China
 gyining@stu.xjtu.edu.cn

Keyan Bai

School of Software Engineering
Xi'an Jiaotong University
 Xi'an 710049, China
 1942363651@stu.xjtu.edu.cn

Zhihao Liu

School of Software Engineering
Xi'an Jiaotong University
 Xi'an 710049, China
 292600050@qq.com

Hua Qu*

School of Software Engineering
Xi'an Jiaotong University
 Xi'an 710049, China

*Corresponding author: qh@mail.xjtu.edu.cn

Abstract—Low Earth orbit (LEO) satellite networks exhibit rapidly changing topology and time-varying traffic hotspots, which makes hop-by-hop routing highly sensitive to local congestion and state staleness. Existing routing methods either rely on global path computation or use plain local observations, while graph-enhanced approaches often focus on generic neighborhood representation rather than direct comparison among candidate next hops. To address this issue, this paper proposes a Local Graph-Aware Routing method (LGAR) for dynamic LEO satellite networks. LGAR organizes the current node, reachable candidate neighbors, and candidate links into a local graph, and then constructs structured action representations through node encoding, relation message extraction, and attention-based context aggregation. The resulting representations are integrated into an off-policy actor-critic framework to support adaptive hop-by-hop routing decisions. Experiments under the hub-inversion setting show that LGAR achieves an average total delay of 47.64 ms and an average queueing delay of 5.81 ms while maintaining a delivery rate of 99.93%. Compared with MATMR, LGAR-NoGraph, and GRLR, LGAR reduces the average total delay by 12.38%, 12.85%, and 30.75%, respectively. Additional scenario, ablation, and scalability results further show that LGAR generalizes beyond the main setting and that its gain mainly comes from local graph modeling and relation-aware action encoding.

Index Terms—LEO satellite networks, hop-by-hop routing, local graph representation, reinforcement learning, hotspot traffic

I. INTRODUCTION

LEO satellite constellations are expected to provide low-latency and wide-area connectivity, but routing in such networks remains challenging because the network state changes continuously with satellite motion and uneven traffic demand [1]. In practical systems, hotspot traffic generated by gateway concentration, regional service bursts, or migrating traffic centers can easily create local queue buildup and delay amplification. As a result, the quality of a hop-by-hop routing decision depends not only on geographic progress toward the

destination but also on instantaneous neighborhood congestion and link conditions around the current forwarding node.

Existing routing methods can be roughly divided into three categories. Traditional shortest-path and global-path-based methods are efficient when topology snapshots and link costs are accurate, but their online adaptability is limited because global state collection and path maintenance become costly under dynamic traffic [2], [3]. Distributed learning-based methods improve scalability and online decision making, yet many of them still rely on plain vectorized local observations, which makes it difficult to explicitly model the structural differences among candidate next hops. Graph-enhanced routing methods further exploit neighborhood information, but they often focus on generic graph representation learning or multi-agent coordination rather than direct action-level comparison among candidate forwarding directions [4]–[6].

For hop-by-hop routing, the decision object at each step is one next-hop action selected from a small candidate set instead of an entire end-to-end path. Therefore, the routing state should be organized in a way that matches the action granularity and the information actually available at decision time. Compared with a complete global-topology representation, a local graph centered on the current node is more suitable for distributed online routing because it directly captures destination guidance, candidate links, and local queueing conditions in the neighborhood where the decision is executed. More importantly, the goal of this work is not merely to use neighborhood information, but to build action-aligned local graph representations so that each candidate forwarding direction can be evaluated under a shared local topology context. This distinguishes LGAR from MATMR-style vectorized local observations and from generic neighborhood graph encoders such as GRLR and GraphPR. This action alignment distinguishes LGAR from generic neighborhood encoders, because each local edge is explicitly treated as one executable forwarding option rather than as an undifferentiated

topological cue.

Based on this observation, this paper proposes a Local Graph-Aware Routing method, abbreviated as LGAR, for dynamic LEO satellite networks. The main contributions of this paper are threefold. First, a local-graph-based state modeling method is developed for hop-by-hop routing, where the current node, candidate neighbors, and candidate links are organized into an action-aligned local graph. Second, a local graph encoding and structured action representation mechanism is designed to improve discriminative comparison among candidate next hops through node encoding, relation message extraction, and attention-based context aggregation. Third, experiments under hub-inversion traffic as well as additional traffic scenarios demonstrate that LGAR reduces end-to-end delay while maintaining a high delivery rate, and the main gain comes from queueing delay reduction. The remainder of this paper presents the problem formulation, the LGAR method, and the experimental validation.

II. PROBLEM FORMULATION

A. Network and Delay Model

We consider a dynamic LEO satellite network represented by a time-varying graph

$$G(t) = (V, E(t)), \quad (1)$$

where V denotes the set of satellites and $E(t)$ denotes the set of available inter-satellite links at time t . A packet is forwarded hop by hop from the source to the destination, and the current forwarding node chooses one reachable neighbor as the next hop at each decision step. In the considered distributed online setting, each node mainly relies on locally observable neighbor and link information rather than a continuously synchronized global network state.

For a packet delivered through a path with H hops, the end-to-end delay is decomposed as

$$D = \sum_{h=1}^H (\tau_h^{\text{queue}} + \tau_h^{\text{tx}} + \tau_h^{\text{prop}}), \quad (2)$$

where τ_h^{queue} , τ_h^{tx} , and τ_h^{prop} denote the queueing, transmission, and propagation delays at hop h , respectively. In dynamic hotspot scenarios, the queueing term is usually the most sensitive to local traffic imbalance and therefore becomes a key indicator for evaluating the quality of hop-by-hop routing decisions.

B. Local-Graph-Based Hop-by-Hop Decision

At decision time, let the packet be located at node v_i . The current node first constructs a local graph centered on itself,

$$G_i^{\text{loc}}(t) = (V_i^{\text{loc}}(t), E_i^{\text{loc}}(t)), \quad (3)$$

with

$$V_i^{\text{loc}}(t) = \{v_i\} \cup \mathcal{N}_i(t), \quad E_i^{\text{loc}}(t) = \{(v_i, v_j) \mid v_j \in \mathcal{N}_i(t)\}, \quad (4)$$

where $\mathcal{N}_i(t)$ is the set of reachable candidate neighbors of v_i at time t . This local graph forms a star-shaped neighborhood

centered on the current node, and each local edge corresponds to one executable forwarding direction. Therefore, the candidate action space is directly aligned with the local graph structure.

Based on the above construction, the routing process is formulated as a Markov decision process

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle. \quad (5)$$

The state at time t is defined as

$$s_t = (G_i^{\text{loc}}(t), \mathbf{X}_i(t), \mathbf{E}_i(t), \mathbf{g}_i(t)), \quad (6)$$

where $\mathbf{X}_i(t)$ denotes the node-feature matrix of the local graph, $\mathbf{E}_i(t)$ denotes the edge-feature matrix, and $\mathbf{g}_i(t)$ denotes destination guidance information. The node features describe node status, destination-related cues, and local load conditions, while the edge features characterize propagation delay and link availability. Hence, the decision state contains not only isolated neighbor attributes but also the structural relations among the current node, candidate neighbors, and candidate links.

The action is to select one candidate neighbor as the next hop,

$$a_t \in \mathcal{A}(s_t) = \mathcal{N}_i(t). \quad (7)$$

After action execution, the packet location, candidate neighbor set, and local graph attributes may all change because of topology evolution, link-state variation, and queue updates. The transition dynamics are captured by $\mathcal{P}(s_{t+1} \mid s_t, a_t)$. Let $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$ denote the immediate routing reward. The goal is to learn a policy that maximizes the discounted cumulative return,

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad \pi^* = \arg \max_{\pi} J(\pi), \quad (8)$$

where $\gamma \in (0, 1)$ is the discount factor and T is the terminal step of packet delivery. In this formulation, hop-by-hop routing is treated as action-level decision making on a dynamic local graph rather than path generation on a static global topology.

III. PROPOSED LGAR

A. Overview

Figure 1 illustrates the overall framework of LGAR. The method consists of three tightly coupled stages: local graph state construction, local graph encoding, and policy learning. For each packet forwarding step, the current node first organizes its local observation into a local graph state centered on itself and its reachable candidate neighbors. The local graph encoder then extracts node-level, relation-level, and context-level features and produces structured representations for all candidate actions. These action representations are further fed into the policy and value networks to support next-hop selection and off-policy training. In this framework, the local graph is not only a compact state description, but also the common basis for candidate-action comparison and value estimation. The following subsections describe the local graph

IV. EXPERIMENTS AND RESULTS

A. Experimental Setup

Experiments are conducted in a Python/PyTorch-based LEO network simulator under a Walker-Delta constellation with altitude 600 km, inclination 53° , 18 orbital planes, 16 satellites per plane, and phasing parameter 3. Three gateway regions, namely Beijing, Luxembourg, and Dubai, are considered. The main setting is a hub-inversion traffic scenario, where dominant gateway-pair demand is periodically reversed and creates time-varying local congestion. The training and evaluation durations are 60 s and 45 s, respectively. The base Poisson traffic rate is 4.8 packets/ms, the normal-packet probability is 0.72, and the two packet sizes are 0.052 for normal packets and 0.016 for small packets.

The proposed method is compared with SPF, GRLR, MATMR, and GraphPR. SPF serves as a traditional global-path heuristic baseline, MATMR represents a local-observation cooperative baseline, and GRLR and GraphPR serve as neighborhood-graph-enhanced learning baselines. All reported evaluation results are averaged over three random seeds. The main evaluation metrics are delivery rate, total delay, and queueing delay. Additional experiments are further organized into multi-scenario generalization, ablation analysis, and scale-extension tests. These added evaluations are intended to separately examine robustness across traffic patterns, the contribution of individual model components, and the practicality of online deployment as the constellation grows.

B. Overall Performance

TABLE 1.
OVERALL PERFORMANCE UNDER HUB-INVERSION EVALUATION.

Method	Delivery (%)	Total (ms)	Queue (ms)
SPF	10.82	79.38	43.30
GRLR	91.56	68.80	17.44
MATMR	99.93	54.37	10.78
GraphPR	99.88	107.00	50.30
LGAR	99.93	47.64	5.81
LGAR-NoGraph	99.93	54.67	12.26

Table 1 reports the averaged delay and delivery metrics under hub-inversion evaluation. LGAR attains an average total delay of 47.64 ms while maintaining a delivery rate of 99.93%. Compared with MATMR, LGAR-NoGraph, and GRLR, the average total delay is reduced by 12.38%, 12.85%, and 30.75%, respectively. In contrast, SPF delivers only 10.82% of packets because shortest-path forwarding cannot react to hotspot-induced queue buildup in time, and GraphPR suffers from both high mean delay and a long tail. Relative to MATMR, the non-substitutable gain of LGAR comes from action-aligned local graph modeling instead of plain vectorized local state, which cuts queueing delay from 10.78 ms to 5.81 ms.

C. Generalization and Ablation Analysis

Table 2 evaluates whether the method generalizes beyond the hub-inversion setting.

TABLE 2.
GENERALIZATION PERFORMANCE UNDER ADDITIONAL TRAFFIC SCENARIOS.

Scenario	Method	Delivery (%)	Total (ms)	Queue (ms)
Hotspot-Shift	SPF	34.18	66.42	28.15
Hotspot-Shift	GRLR	97.61	57.28	12.03
Hotspot-Shift	MATMR	99.94	49.86	8.47
Hotspot-Shift	GraphPR	99.90	88.74	34.62
Hotspot-Shift	LGAR	99.96	43.91	4.29
Uniform-Midload	SPF	84.36	61.12	18.96
Uniform-Midload	GRLR	98.88	52.73	9.36
Uniform-Midload	MATMR	99.95	47.82	7.01
Uniform-Midload	GraphPR	99.91	73.55	26.08
Uniform-Midload	LGAR	99.96	44.38	4.86

LGAR remains the best method in both additional scenarios. Under hotspot-shift traffic, LGAR reduces the total delay from 49.86 ms to 43.91 ms compared with MATMR and lowers the queueing delay from 8.47 ms to 4.29 ms. Under uniform-midload traffic, LGAR still achieves the lowest total and queueing delays, which indicates that the gain is not limited to a single hotspot-redistribution pattern. These results also show that LGAR is not simply a heavier version of GRLR or GraphPR; its benefit persists across different traffic regimes because the local graph is explicitly organized for candidate next-hop comparison.

TABLE 3.
UNIFIED ABLATION RESULTS OF LGAR VARIANTS UNDER THE HUB-INVERSION SCENARIO.

Method	Delivery Rate (%)	Total Delay Mean (ms)	Queue Delay Mean (ms)
LGAR	99.93	47.64	5.81
LGAR-NoGraph	99.93	54.67	12.26
LGAR-NoRelation	99.92	52.84	10.31
LGAR-NoAttention	99.93	50.98	8.37

Table 3 consolidates all LGAR variants into a single ablation comparison and isolates the contribution of different model components. Removing local graph modeling causes the largest performance drop, which verifies that action-aligned local graph state construction is the primary source of improvement. Removing relation messages still increases the total delay from 47.64 ms to 52.84 ms, while removing attention aggregation increases it to 50.98 ms. Therefore, both relation-aware action encoding and attention-based context aggregation contribute to the final gain, while the largest reduction in queueing delay is obtained only when all components are jointly retained.

TABLE 4.
DEPLOYMENT-ORIENTED COMPLEXITY AND RUNTIME COMPARISON.

Method	State Dim.	Params (M)	Memory (MB)	Route Time (ms/call)
GRLR	103	0.67	2.56	1.77 ± 0.11
MATMR	103	4.49	17.14	1.60 ± 0.07
GraphPR	103	0.36	1.38	1.88 ± 0.02
LGAR	103	1.11	4.23	1.59 ± 0.10

TABLE 5.
SCALABILITY OF LGAR UNDER DIFFERENT CONSTELLATION SIZES.

Constellation Size	Delivery Rate (%)	Total Delay Mean (ms)	Queue Delay Mean (ms)	Route Time (ms/call)
288	99.93	47.64	5.81	0.24
576	99.94	47.18	5.52	0.25
864	99.94	46.97	5.33	0.26

D. Scalability and Deployment Cost

Table 4 summarizes the deployment-oriented statistics of the learned routing policies. The reported route time is measured on the CUDA-based simulator host as `route_time_s/route_calls` and is reported as mean $\pm$ std over profiling epochs 2–6 after excluding warm-up epoch 1. LGAR requires 1.11M inference parameters and about 4.23 MB storage, which remains moderate compared with MATMR. Observation construction and action masking remain below 0.10 and 0.01 ms/call, respectively, across the profiled methods. Although MATMR has a similar route-time mean, Table 1 shows that LGAR achieves much lower queueing delay. Compared with GRLR and GraphPR, the gain of LGAR is also not explained by using a larger or slower online model; instead, the ablation results indicate that the main advantage comes from relation-aware action encoding on top of local graph state modeling.

Table 5 further examines whether the proposed method remains effective when the constellation becomes larger. When the constellation size increases from 288 to 864 satellites, LGAR maintains a delivery rate around 99.94%, while the total delay and queueing delay remain stable and even slightly decrease. The average route time only changes from 0.24 ms/call to 0.26 ms/call, which suggests that, under a fixed four-direction action space, the proposed local graph-aware policy remains computationally efficient as the constellation grows.

V. CONCLUSION

This paper presented LGAR, a local graph-aware hop-by-hop routing method for dynamic LEO satellite networks. By organizing the current node, candidate neighbors, and candidate links into an action-aligned local graph and learning

structured action representations, LGAR improves candidate next-hop discrimination in dynamic routing environments. Experimental results under hub-inversion traffic, additional traffic scenarios, and larger constellation sizes show that LGAR maintains a high delivery rate while reducing end-to-end total delay, with the main gain coming from queueing delay reduction. Future work can extend this framework to larger constellations, failure-aware routing, and multi-service traffic scenarios.

REFERENCES

- [1] C. Westphal, L. Han, and R. Li, “LEO satellite networking relaunched: Survey and current research challenges,” *ITU Journal on Future and Evolving Technologies*, vol. 4, no. 4, pp. 711–744, 2023, doi: 10.52953/LWXC1928.
- [2] E. Ekici, I. F. Akyildiz, and M. D. Bender, “A distributed routing algorithm for datagram traffic in LEO satellite networks,” *IEEE/ACM Transactions on Networking*, vol. 9, no. 2, pp. 137–147, 2001, doi: 10.1109/90.917071.
- [3] Q. Chen, L. Yang, Y. Zhao, Y. Wang, H. Zhou, and X. Chen, “Shortest path in LEO satellite constellation networks: An explicit analytic approach,” *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 5, pp. 1175–1187, 2024, doi: 10.1109/JSAC.2024.3365873.
- [4] S. Zhang, A. Liu, C. Han, X. Xu, X. Liang, K. An, and Y. Zhang, “GRLR: Routing with graph neural network and reinforcement learning for mega LEO satellite constellations,” *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3225–3237, 2025, doi: 10.1109/TVT.2024.3471658.
- [5] Y. Ran, Y. Ding, S. Chen, J. Lei, and J. Luo, “Fully-distributed dynamic packet routing for LEO satellite networks: A GNN-enhanced multi-agent reinforcement learning approach,” *IEEE Transactions on Vehicular Technology*, vol. 74, no. 3, pp. 5229–5234, 2025. Available: <https://ieeexplore.ieee.org/document/10755127>
- [6] X. Chen, Z. Ji, S. Wu, H. Jia, A. Xiao, and C. Jiang, “A distributed routing algorithm for LEO satellite networks: A multiagent transformer-MIX learning approach,” *IEEE Internet of Things Journal*, vol. 12, no. 11, pp. 15748–15763, 2025, doi: 10.1109/JIOT.2025.3530919.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870. Available: <https://proceedings.mlr.press/v80/haarnoja18b.html>
- [8] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proc. International Conference on Machine Learning (ICML)*, 2018, pp. 1587–1596. Available: <https://proceedings.mlr.press/v80/fujimoto18a.html>